

Assignment -3:

Code

```
import java.util.Scanner;
```

```
class InvalidMarksException extends Exception {  
    public InvalidMarksException(String msg){super(msg);}  
}
```

```
class Student {  
    int rollNumber;  
    String studentName;  
    int[] marks;
```

```
    public Student(int rollNumber, String studentName, int[] marks)  
        throws InvalidMarksException {  
        this.rollNumber = rollNumber;  
        this.studentName = studentName;  
        if (marks == null || marks.length != 3) {  
            throw new IllegalArgumentException("Exactly 3 arguments required");  
            this.marks = new int[3];  
            System.arraycopy(marks, 0, this.marks, 0, 3);  
            validateMarks();  
        }  
    }
```

```
    public void validateMarks() throws InvalidMarksException {  
        for (int i = 0; i < marks.length; i++) {  
            int m = marks[i];  
            if (m < 0 || m > 100) throw new  
                throw new InvalidMarksException  
                ("Invalid marks for subject");  
        }  
    }
```

//_

```
public double calculateAverage() {  
    int s = 0;  
    for (int m: marks) s += m;  
    return s / 3.0; }  

```

```
public String getResultStatus() {  
    for (int m: marks) { if (m < 40) { return "fail"; } }  
    return "Pass"; }  

```

```
public int getRollNumber() {  
    return rollNumber; }  

```

```
public void displayResult() {  
    System.out.println("Roll No : " + rollNumber);  
    System.out.println("Student Name : " + studentName);  
    System.out.println("Marks :");  
    for (int i = 0; i < marks.length; i++) {  
        System.out.println(marks[i] + (i < marks.length - 1 ? " " : " "));  
        System.out.println();  
        System.out.println("Average : " + calculateAverage());  
        System.out.println("Result : " + getResultStatus()); } }  

```

```
class ResultManager {  
    static final int MAX_STUDENTS = 100;  
    Student[] students;  
    int studentCount;  
    Scanner scanner;  

```

//_

```
public ResultManager() {  
    students = new Student[MAX-STUDENTS];  
    studentCount = 0;  
    Scanner = new Scanner(System.in);  
}
```

```
String readLineInt(String prompt) {  
    while (true) {  
        System.out.print(prompt);  
        String line = Scanner.nextLine();  
        try {  
            return Integer.parseInt(line);  
        }  
        catch (Exception e) {  
            System.out.println("Enter valid Integer");  
        }  
    }  
}
```

```
String readLine(String prompt) {  
    while (true) {  
        System.out.print(prompt);  
        String line = Scanner.nextLine();  
        if (line != null) {  
            if (!line.isEmpty()) return line;  
            System.out.println("Input cannot be empty");  
        }  
    }  
}
```

```
int findStudent(int roll) {  
    for (int i = 0; i < studentCount; i++) {  
        if (student[i].getRollNumber() == roll)  
            return i;  
    }  
    return -1;  
}
```



```

public void addStudent() {
    try { int roll = readInt("Enter Roll Number:");
        if (findStudent(roll) != -1) {
            sout("Student with roll "+ roll + " exist"); return; }
        String name = readLine("Enter Student Name");
        while (true) {
            int [] marks = new int [3];
            for (int i = 0; i < 3; i++) { marks[i] =
                readInt("Enter marks " + (i + 1) + " (0-100)"); }
            try {
                Student s = new Student(roll, name, marks);
                students[studentCount++] = s;
                System.out.println("Student added");
                break;
            } catch (InvalidMarksException ime) {
                System.out.println("Error: " + ime.getMessage());
                System.out.println("Re-enter marks"); } }
        }
    } catch (Exception e) {
        sout("Unexpected error" + e.getMessage()); }
}

```

```

public void showDetail() {
    int roll = readInt("Enter Roll No to search");
    int idx = findStudent(roll);
    if (idx == -1) System.out.println("Student not found");
    else students[idx].displayResult(); }

```

//_

```

public void mainMenu() {
    boolean exit = false;
    try {
        while (!exit) {
            System.out.println("----- Student Result Management");
            System.out.println("1. Add Student");
            System.out.println("2. Show Details");
            System.out.println("3. Exit");

            int choice = readInt("Enter choice");
            switch (choice) {
                case 1: addStudent(); break;
                case 2: showStudentDetails(); break;
                case 3: System.out.println("Exiting Thanks"); {
                    exit = true; break;
                }
                default: System.out.println("Invalid choice"); } }

        } finally { if (scanner != null) {
            try { scanner.close(); }
            catch (Exception ignored) {} }
            System.out.println("Scanner closed"); } }
    }

```

```

public static void main(String[] a) {

```

```

    ResultManager rm = new ResultManager();
    rm.mainMenu(); }

```